CAN DATABASE / ENGINEERING TOOL

DBC Editor

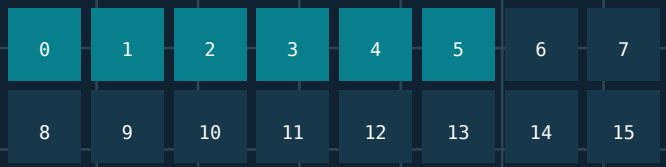
User Guide

Open, create, validate, and download CAN and CAN FD databases.

- Core DBC concepts
- Editor interface and workflow
- Message and signal fields
- Intel / Motorola bit layout
- Value descriptions and multiplexing
- Validation, export, and field verification

Version 1.0 | 26 July 2026

BO_ / SG_



bulentturk.com

TECHNICAL GUIDE

What does a DBC file define?

Rules that convert raw CAN data into engineering values

A DBC is a text-based database containing decoding rules such as CAN ID, payload length, signal bits, byte order, sign, factor, offset, and unit.

CORE DBC STATEMENTS

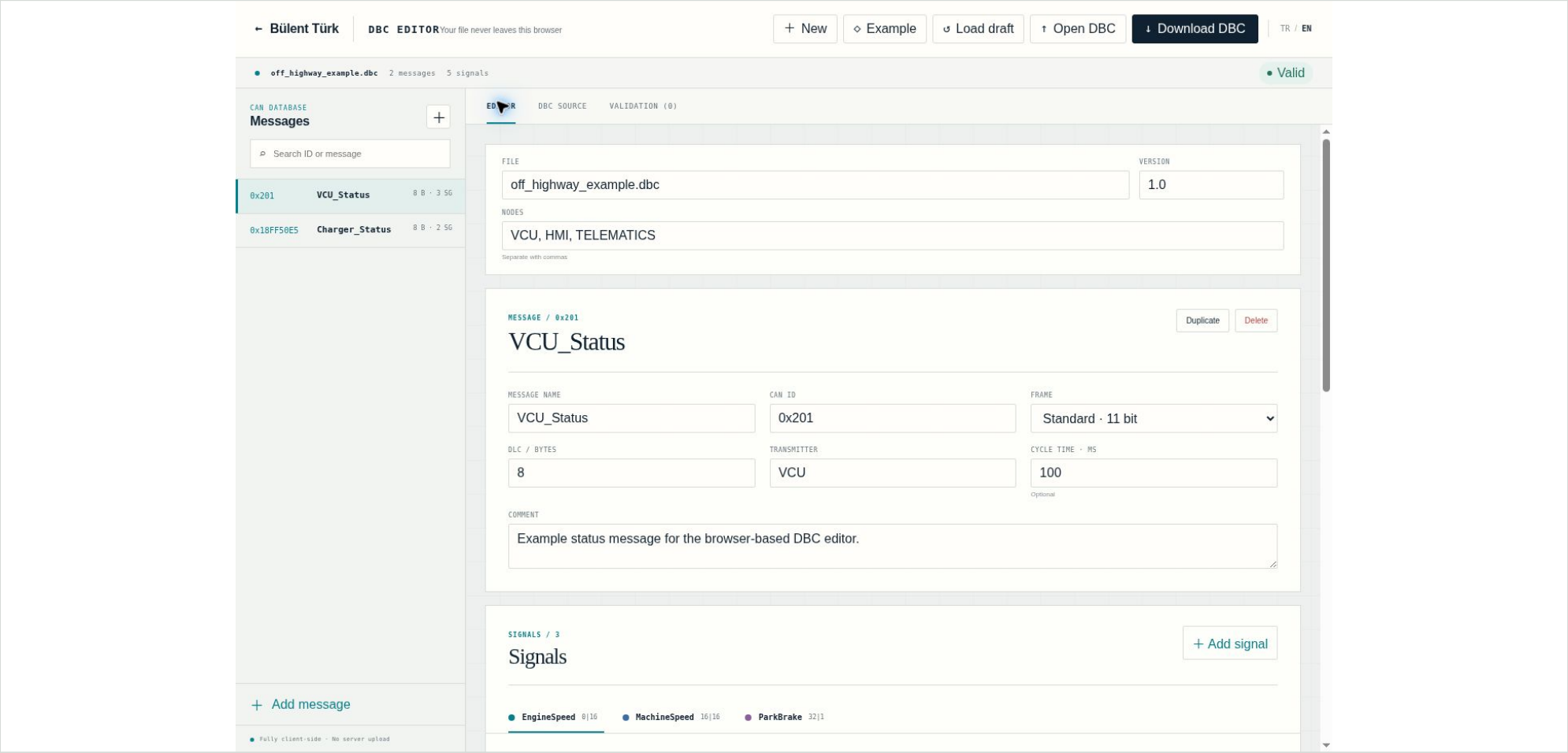
```
BO_ 513 VCU_Status: 8 VCU
 SG_ EngineSpeed : 0|16@1+ (0.125,0) [0|8031.875] "rpm" HMI
```

Important: A DBC does not produce data or connect to a CAN bus. It only defines how existing raw CAN frames should be interpreted.

TECHNICAL GUIDE

Editor tour

The main controls are available in one workspace



1 Top toolbar: New, Example, Load draft, Open DBC, Download DBC, and PDF Guide. 2 Left panel: message list and search. 3 File metadata: name, version, nodes. 4 Message card. 5 Signal card. 6 Bit layout and validation tab.

TECHNICAL GUIDE

Two working paths

Edit an existing DBC or start from scratch

A / OPEN AN EXISTING FILE

- Use Open DBC or drag a .dbc file onto the editor.
- Review CAN IDs and message names in the message list.
- Select the required message and signal, then edit the fields.
- Resolve issues reported in the Validation tab.
- Save the result with Download DBC.

B / CREATE A NEW DBC

- Press New and confirm the clean database.
- Enter the file name, version, and node names.
- Use Add message to define CAN ID, frame type, and DLC.
- Use Add signal to define bit position, byte order, and scaling.
- Validate, download, and test with raw CAN data.

Draft and privacy

- Changes are automatically saved as a local browser draft.
- Load draft restores the latest work from the same browser.
- The imported DBC is not uploaded; processing is client-side.
- Clearing browser data may remove the local draft. Download versions regularly.
- The maximum file size is 20 MB.

TECHNICAL GUIDE

Message settings

Each BO_ entry defines one CAN frame

FIELD	PURPOSE	EXAMPLE
Message name	Unique DBC identifier; do not use spaces.	VCU_Status
CAN ID	Hexadecimal or decimal input is accepted.	0x201 / 513
Frame	Standard 11-bit or Extended 29-bit.	Standard
DLC / bytes	Payload length; Classic CAN is commonly 0-8.	8
Transmitter	Node that publishes the message.	VCU
Cycle time	Optional GenMsgCycleTime value.	100 ms
Comment	Message purpose; written as CM_ BO_.	VCU status

- Standard CAN ID range: 0x000 - 0x7FF.
- Extended CAN ID range: 0x00000000 - 0x1FFFFFFF.
- Typical CAN FD payload lengths: 0-8, 12, 16, 20, 24, 32, 48, 64 bytes.
- CAN ID and message name must be unique within the selected frame type.

```
BO_ 513 VCU_Status: 8 VCU
```

When saving, the editor automatically converts a 29-bit ID to the DBC extended-ID representation.

TECHNICAL GUIDE

Signal settings

Each SG_ entry defines one value in the payload

FIELD	PURPOSE	EXAMPLE
Signal name	Unique DBC identifier within the message.	EngineSpeed
Start bit	Start position in DBC bit numbering.	0
Length	Number of bits in the signal.	16 bits
Byte order	Intel little-endian or Motorola big-endian.	Intel
Signedness	Unsigned or two's-complement signed.	Unsigned
Factor / Offset	Converts raw data to a physical value.	0.125 / 0
Min / Max / Unit	Engineering limits and unit metadata.	0 / 8031.875 rpm

```
Physical value = Raw value x Factor + Offset

0x1388 = 5000 raw  ->  5000 x 0.125 + 0 = 625 rpm
```

Min and max are validation/metadata fields; they do not clamp the raw value automatically.

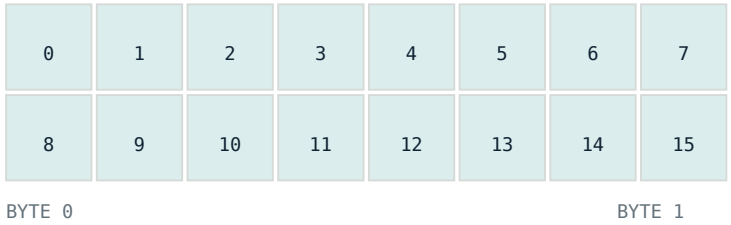
TECHNICAL GUIDE

Bit layout and byte order

The bit traversal changes even when the start-bit number looks similar

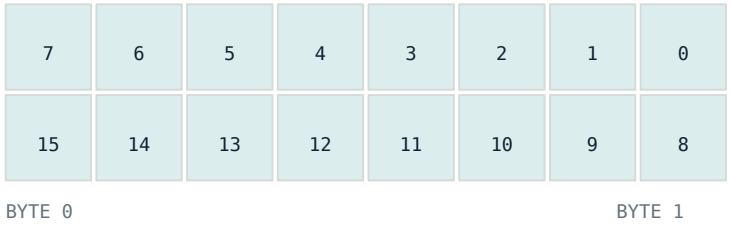
INTEL / LITTLE ENDIAN (@1)

The start bit is the least significant bit (LSB). In multi-byte values, the least significant byte comes first.



MOTOROLA / BIG ENDIAN (@0)

In DBC syntax, the start bit is the most significant bit (MSB). Bits progress in a sawtooth pattern.



! The editor uses DBC-syntax start bits directly. Another GUI may display Motorola start bits differently; always inspect the exported DBC statement.

● Red/conflicting cells indicate that two signals occupy the same payload bit.

TECHNICAL GUIDE

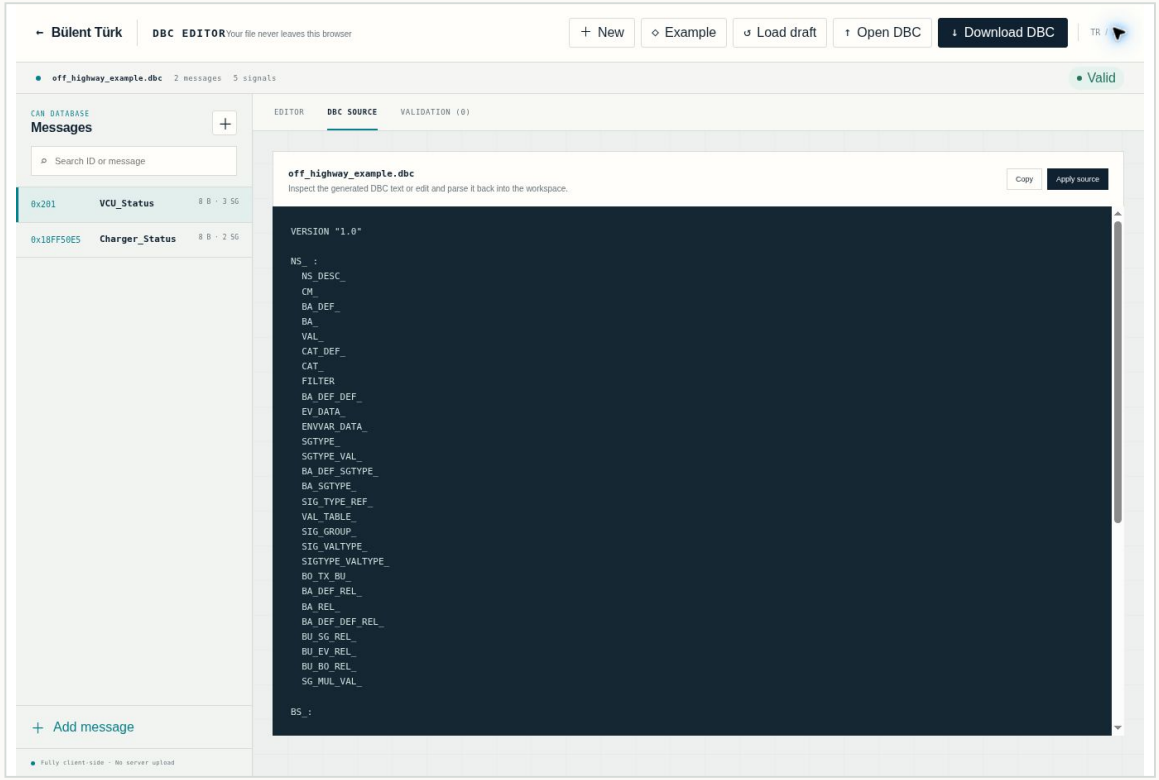
Value descriptions, multiplexing, and DBC source

Make state signals readable and inspect advanced statements as text

- Value descriptions map raw state codes to readable labels.
- Example: 0 = Released, 1 = Applied.
- In the Multiplex field, M marks the multiplexer; m0, m1… select a branch.
- Unsupported custom DBC statements are preserved, but not every advanced construct is editable through visual fields.

```
VAL_ 513 ParkBrake 0 "Released" 1 "Applied" ;

SG_ Mode M : 0|4@1+ ...
SG_ Speed m1 : 8|16@1+ ...
```

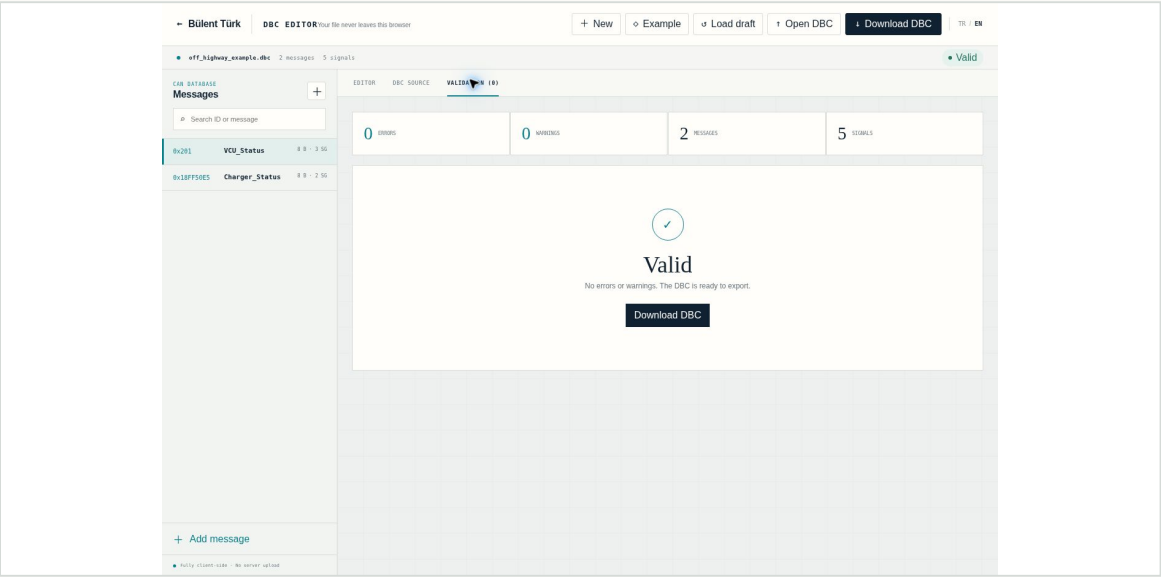


Use the DBC Source tab to inspect or edit the generated text, then parse it back with Apply source.

TECHNICAL GUIDE

Validation and export

Resolve syntax issues, then perform a separate field validation



VALIDATION CODES

`DUPLICATE_MESSAGE_ID`
The same CAN ID is assigned more than once.

`SIGNAL_OUT_OF_BOUNDS`
A signal extends beyond the DLC.

`SIGNAL_OVERLAP`
Signals share the same payload bit.

`INVALID_FACTOR`
Factor is invalid or zero.

`MIN_GREATER_THAN_MAX`
Minimum is greater than maximum.

EXPORT CHECKLIST

- Open Validation and reduce the error count to zero.
- Click an issue to return to the affected message or signal.
- When the database is Valid, use Download DBC.
- Reopen the downloaded file and compare message/signal counts.

A Valid result does not prove that the signal definition matches the real CAN data.

TECHNICAL GUIDE

Worked example and final checks

Decode a VCU_Status frame from raw bytes to physical values

```
CAN ID 0x201
DATA  88 13 10 27 01 00 00 00
```

EngineSpeed

Bytes 0-1

`0x1388 = 5000`

`5000 x 0.125 = 625 rpm`

MachineSpeed

Bytes 2-3

`0x2710 = 10000`

`10000 x 0.01 = 100 km/h`

ParkBrake

Byte 4, bit 0

`1`

`VAL_: 1 = "Applied"`

Before release or handover

- Does CAN ID and Standard/Extended selection match the trace?
- Are DLC and start-bit/length within the payload?
- Was Intel/Motorola verified against supplier documentation and test data?
- Are signedness, factor, offset, and unit correct?
- Were VAL_ descriptions added for state signals?
- Was the DBC opened in another reader/decoder and tested on raw CAN data?
- Are changes under revision control?

Tool boundary

This editor creates DBC files and performs structural checks. It does not provide live CAN monitoring, hardware access, PGN/SPN verification, or automatic validation of real signal semantics.

REFERENCES AND LINKS

DBC Editor
bulentturk.com/dbc-editor/